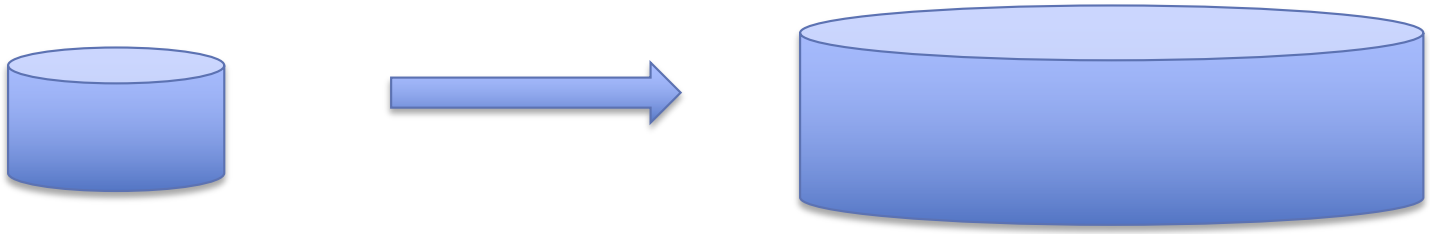
Large Scale Data Processing

Why Distributed Data Processing

- Hardware:
  - CPU speed does not increase
  - Instead: multicore
- Commodity clusters
  - Easy access to 1000 of nodes through cloud computing
  - Much cheaper than large mainframe
- Big Data
  - Astronomy: high-resolution, high-frequency sky surveys
  - Medicine: digital records, MRI, ultrasound
  - Biology: sequencing data
  - User behavior data: click streams, search logs, …
    - Google and Facebook, but also Walmart and co…

Distribution and Performance

- Traditionally: **scale-up**
  - Improve performance by buying larger machine



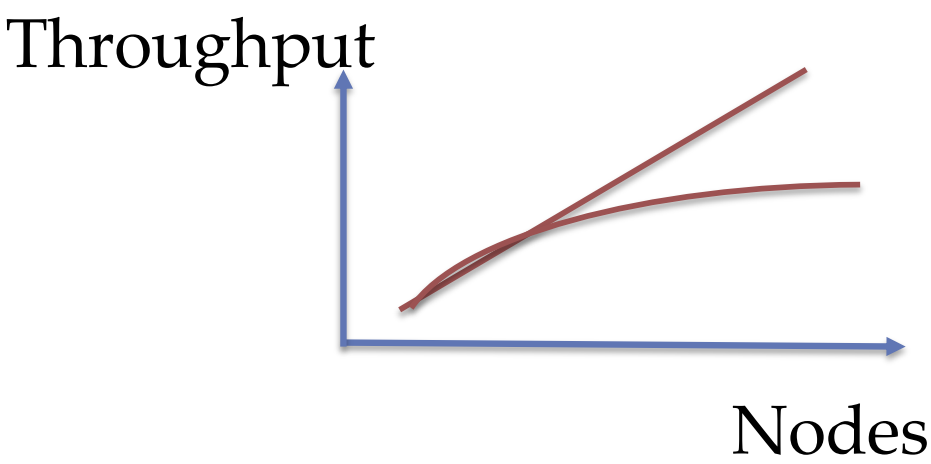
- Distribution: **scale-out**
  - Improve performance through parallel / distributed execution

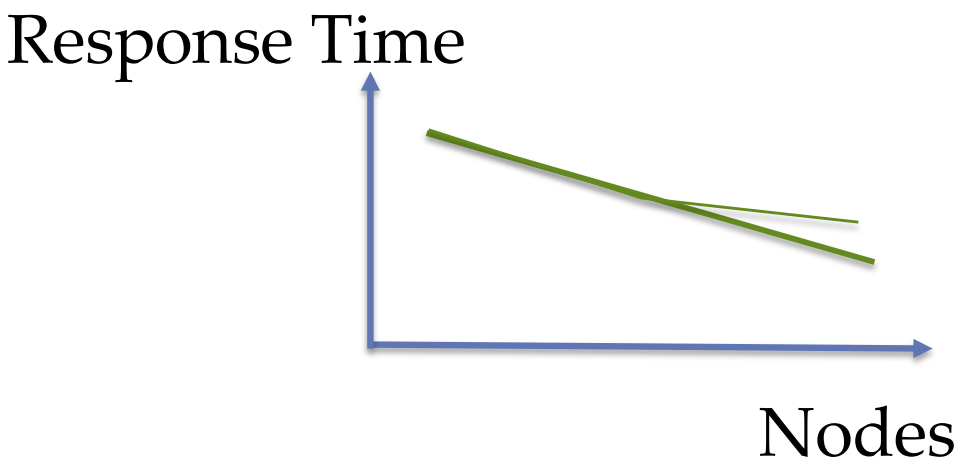
Distribution and Performance

- Performance metrics:
  - Throughput: transactions/queries per time unit
    - The higher the better
    - Important for Online Transaction Processing (OLTP)
      - workload of short, update intensive queries, such as day-to-day banking, flight reservations etc.
  - Response time: time for execution of an individual transaction/query
    - The smaller, the better
    - Important for Online *Analytical* Processing (OLAP)
      - workload of complex queries, mainly read-only

Speedup

- Speedup (the data size remains the same)
  - More nodes → more throughput and/or lower response time





- Non-linear Speedup
  - Startup costs
  - Coordination costs
  - Communication costs
  - Skew (equal distribution of load not possible)

Scaleup

- Scaleup (the data size increases)*:
  - More nodes → have same throughput / same response time despite more data

- Limits to scaleup:
  - Data distribution overhead
  - Non - parallelizable operations
    - aggregation
  - Communication costs
  - Skew (equal distribution of data not possible)

*note the different use of scaleup compared to slide 3 (happens often: same term but different meaning, or different terms but same meaning – always read the fine-print)

Parallel Relational Database Systems

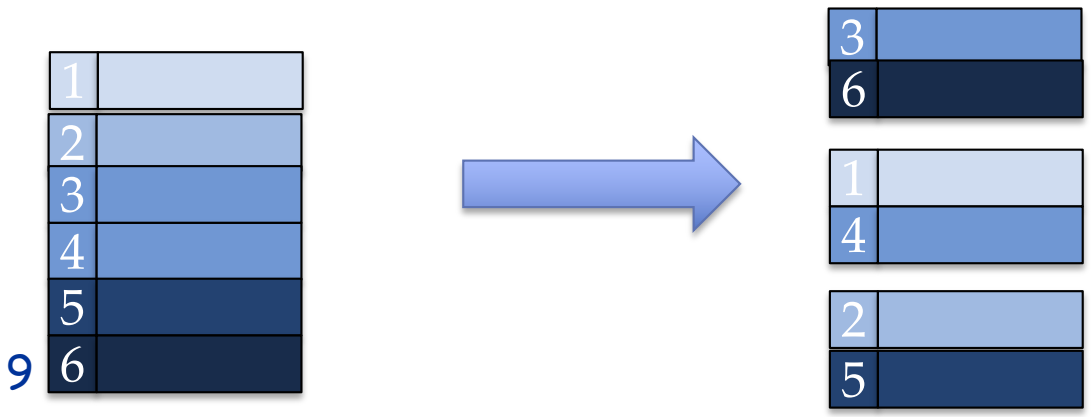
- A lot of DBS technology developed in 90s.
- Good understanding of distributed execution of relational algebra queries
- Sophisticated and optimized operators (such as distributed join operators…)
- Expensive and specialized: Oracle, Teradata,…

Parallel Query Evaluation

- Inter-query parallelism
  - Different queries run in parallel on different processors; each query is executed sequentially
- Inter-operator parallelism
  - Different operators within same execution tree run on different processors
  - Pipelining leads to parallelism
- Intra-operator parallelism
  - A single operator (e.g., scan, join) runs on many processors
  - Topic of this lecture

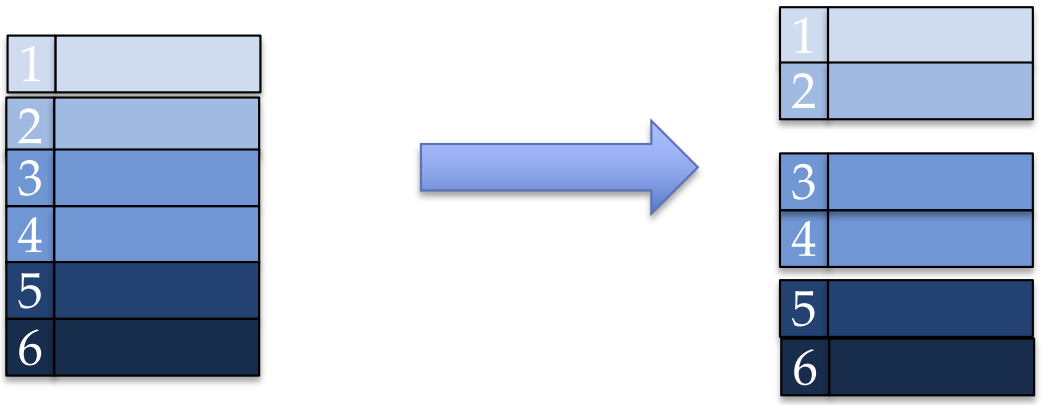
Horizontal Data Partitioning

- Data
  - Large table R(K, A, B C)
  - Key-value store KV(K, V)
- Goal
  - partition into chunks $C_1, C_2, \ldots C_n$ of records stored at n nodes
- Hash partition on attribute X (for Key-value store that's always K):
  - Record r goes to chunk i, according to hash function
  - Example hash-function: H = r.X mod n



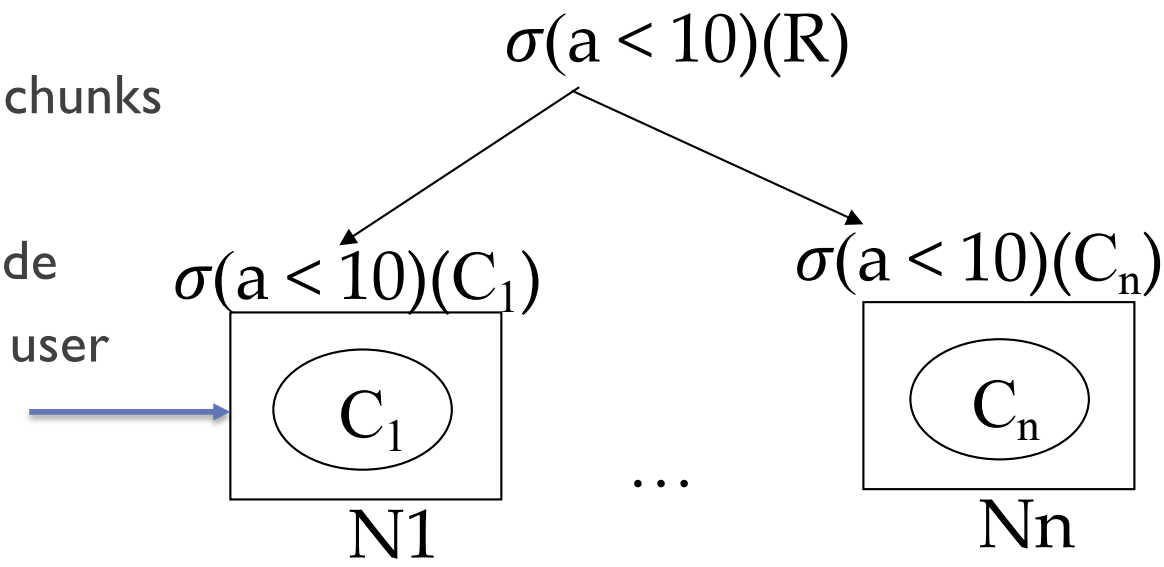
Horizontal Data Partitioning II

- Range partition on attribute X (for Key-value store that's always K):
  - Partition range of X into: $-\infty = v_1 < v_2 < \ldots < v_{n-1} = \infty$
  - Record r goes to chunk i, if $v_{i-1} <= r.X < v_i$



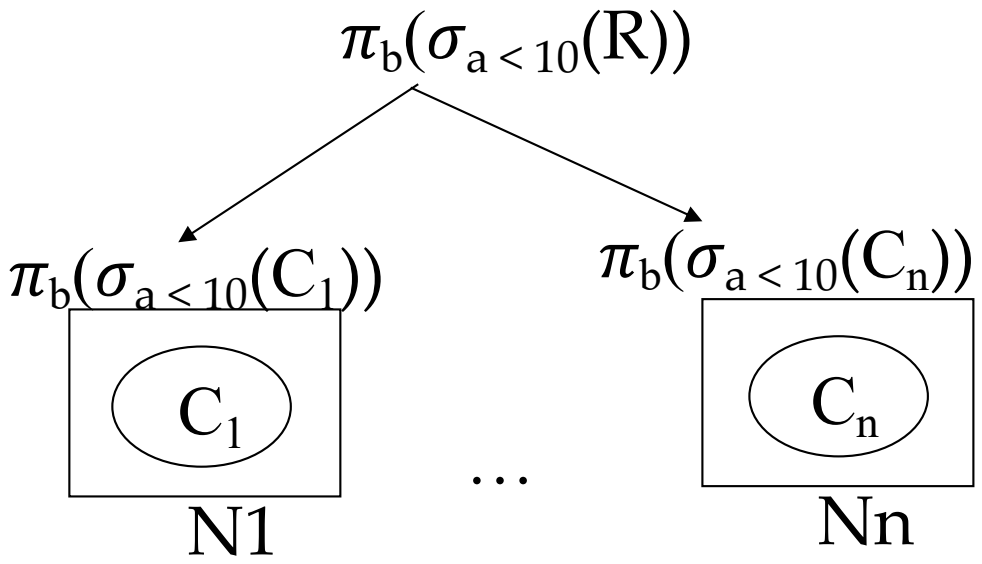
Example parallel operator: selection

- Execution path
  - Push selections to nodes with chunks
  - Execute locally at each node
  - Send result to coordinating node
  - Assemble result and return to user
- Basics:
  - move and split of operators
  - Parallel operator execution
  - Transfer of records across nodes
  - Merge / post-processing at coordinating node
- Goals: Minimize CPU/IO/communication
  - Equal (non skewed) distribution of processing and I/O costs
    - Partition data equally
  - Keep communication costs low: execute locally, minimize transfer

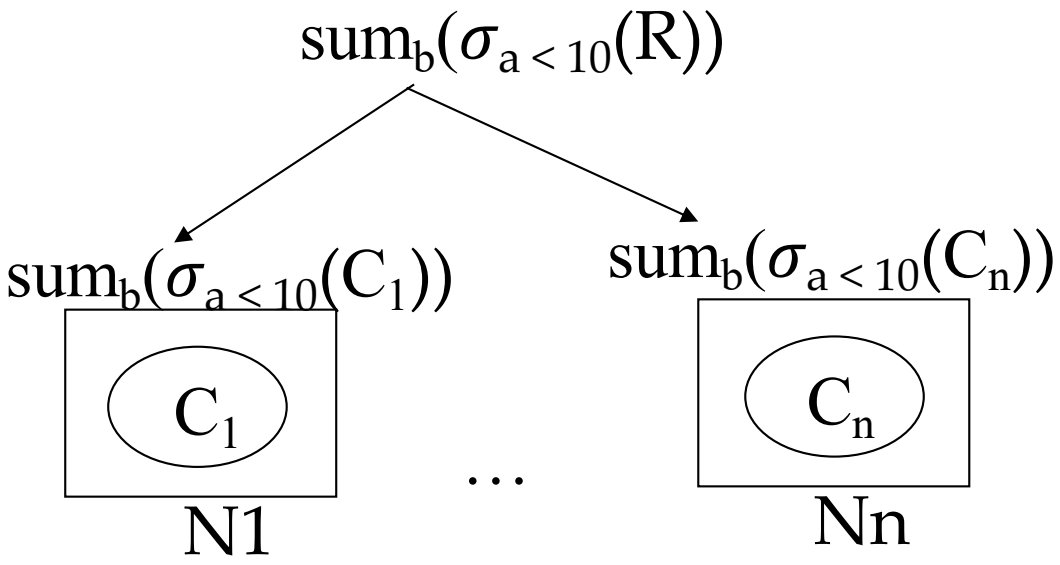


What about?

- Projection?
  - Can also be done locally at each node

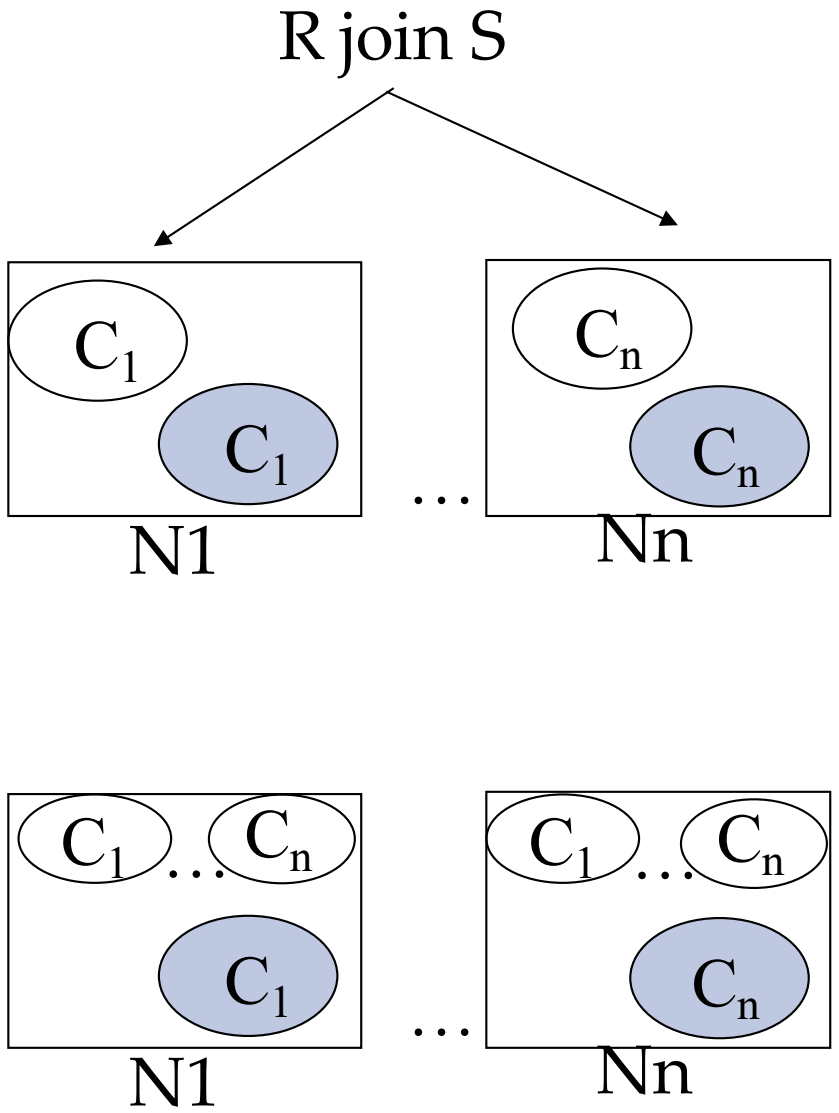


- Aggregation?
  - Post-processing
  - We have to sum the sums
  - Different post-processing needed depending on aggregation function



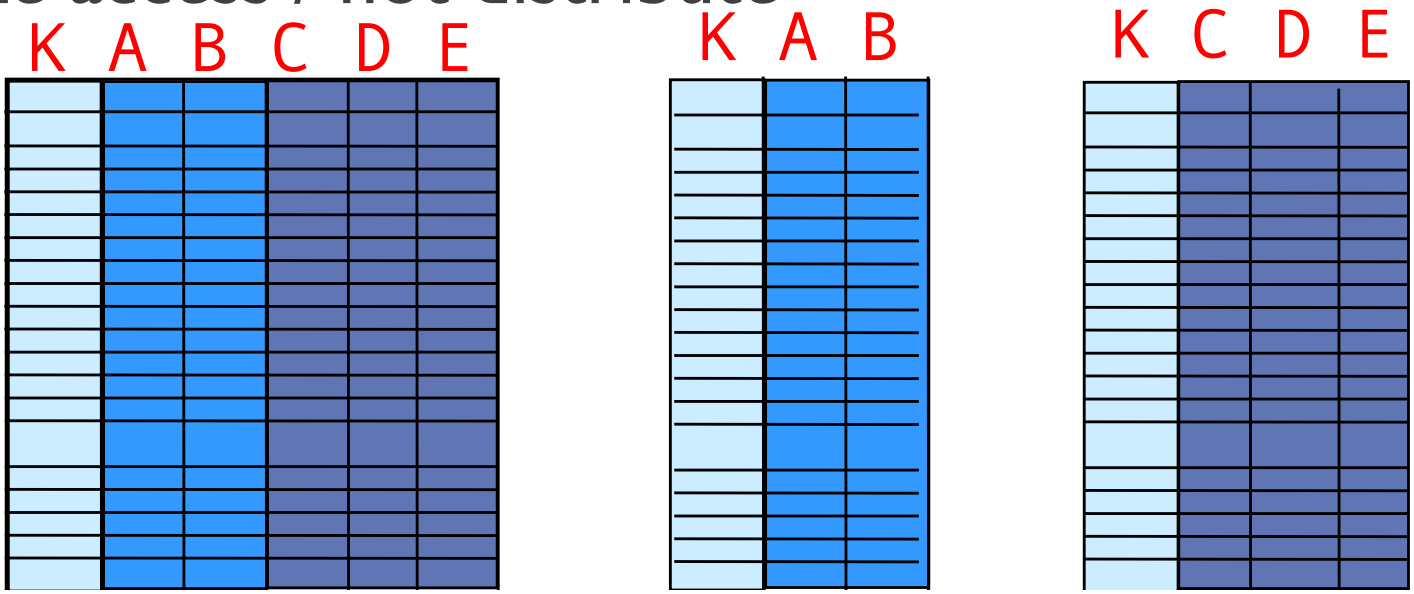
Join?

- Both relations are partitioned
  - Cannot be done locally
  - We have to move data between nodes so that each node can do part of the join
  - Simplest:
    - Replicate the chunks of smaller relation everywhere
    - Join locally with the chunk of the other relation
    - Merge results



Note: Vertical Data Partitioning

- Column-Stores
- Data: relations R(K, A, B, C, D, E)
- Partition into RAB(K, A, B), RCDE(K, C, D, E)
- Query
  - *SELECT A from R where B > 50*
- *Query* only needs to access partition RAB
- Localize access / not distribute



Vertical Data Partitioning

- Why is the key replicated?

```
SELECT * FROM R
```

Equals

```
SELECT * FROM RAB, RCDE
WHERE RAB.K = RCDE.K
```

Map-reduce

- General-purpose distributed computing framework
- Can be applied to many types of queries; non-relational and relational
- Developed by Google; open-source version ***Hadoop*** developed by Yahoo led to quick success
- One initiative within the NoSQL movement

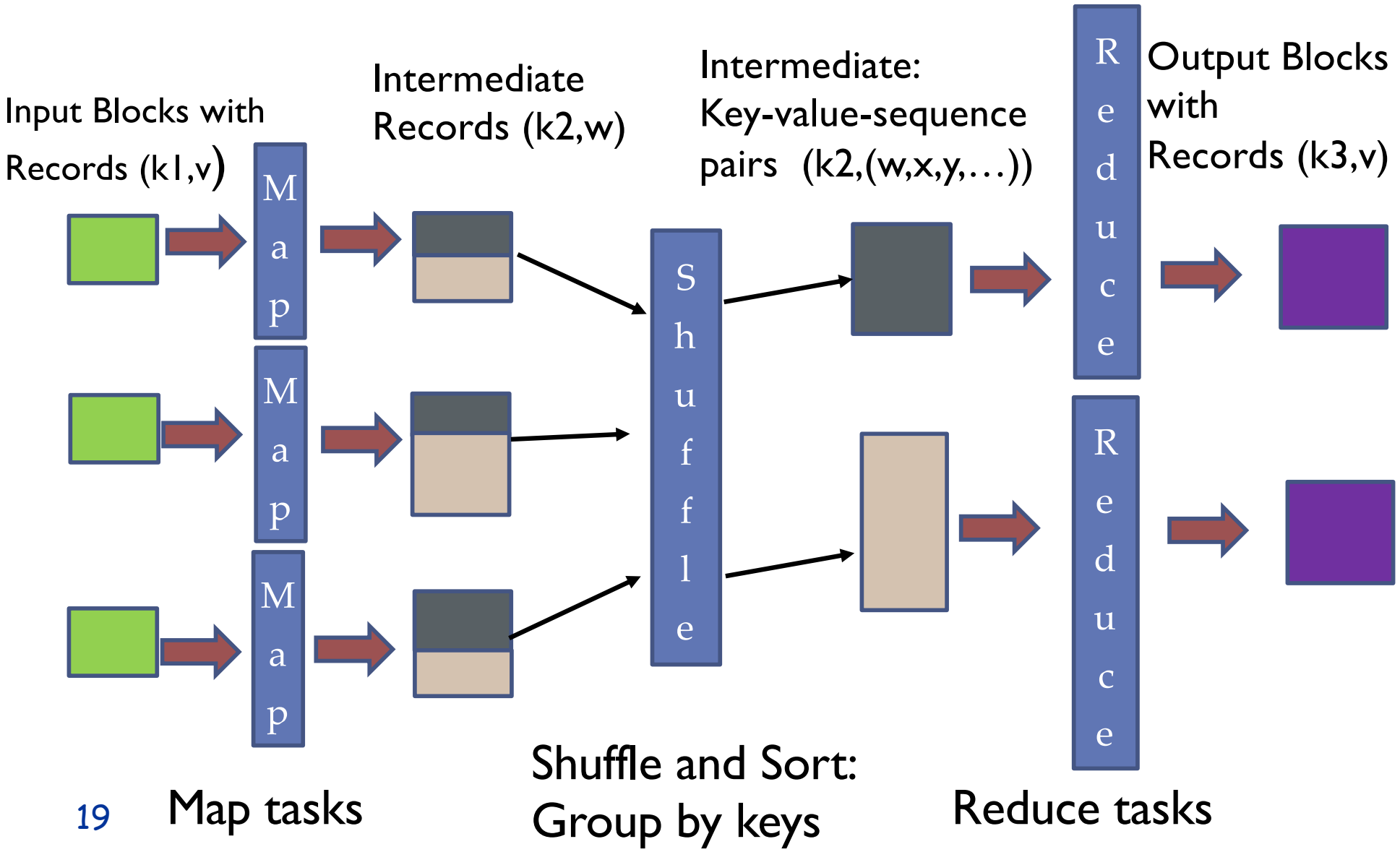
Data Processing at Massive Scale

- Massive Scale
  - Petabytes of data
  - 100s, 1000s, 10000s of servers
  - Many hours
- Failure becomes an issue
  - If medium-time-between failure is 1 year
  - Then 10000 servers have one failure / hour
  - Query execution must succeed even if individual nodes fail

Map-reduce

- High-level programming model AND implementation for large-scale parallel data processing
- Programming model
  - Distribute data and each node reads data records one by one (key-value pairs)
  - ***Map tasks:*** extract something interesting from records and output a new set of data records (key-value pairs)
  - Shuffle and sort (same key to same reduce task)
  - ***Reduce tasks:*** aggregate, summarize, filter
  - Write the results

Overview


Input Blocks with Records (k1,v)
Map
Map
Map
Intermediate Records (k2,w)
Shuffle
Intermediate: Key-value-sequence pairs (k2,(w,x,y,…))
Reduce
Reduce
Output Blocks with Records (k3,v)
Map tasks
Shuffle and Sort: Group by keys
Reduce tasks

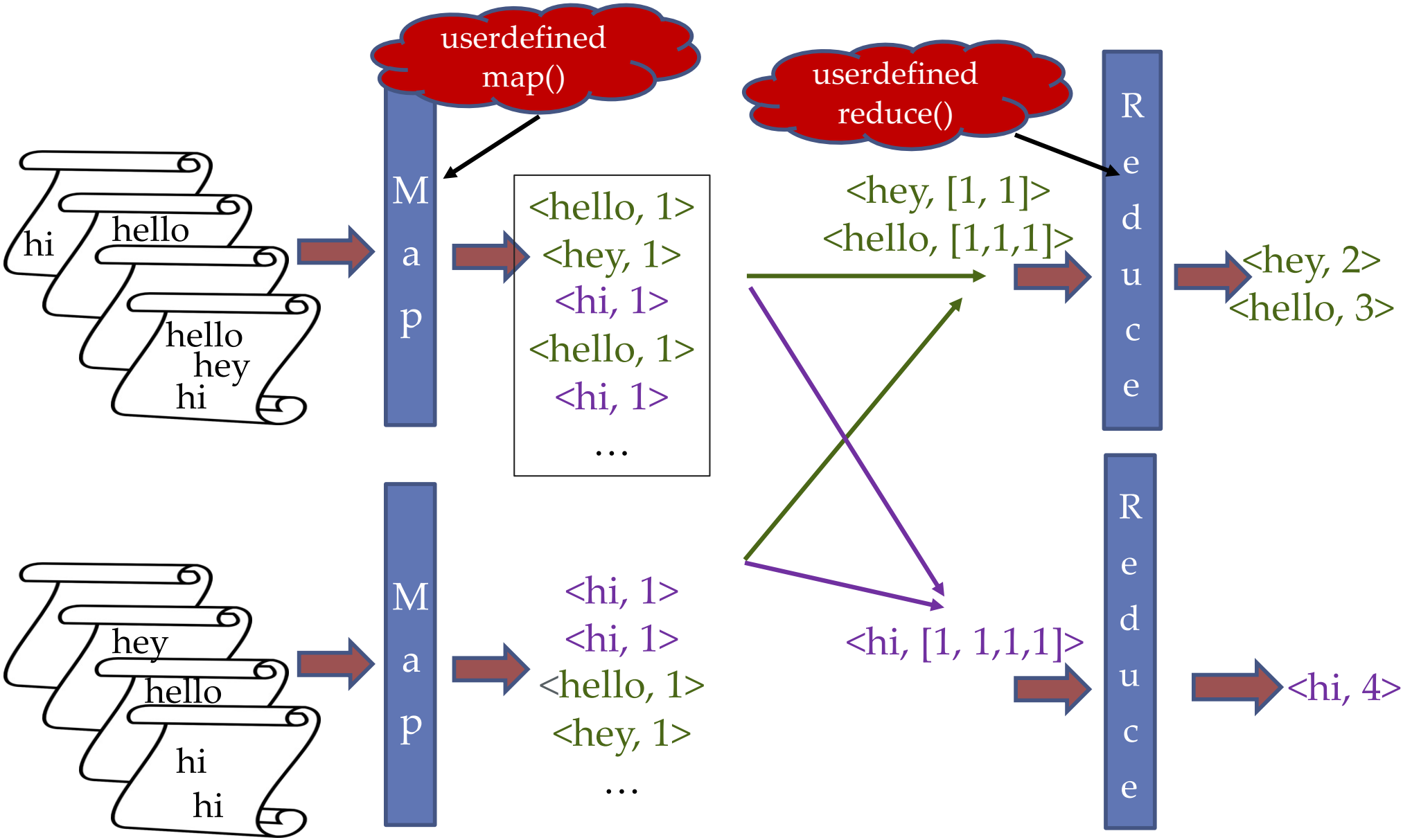
Overview

- Input and output considered key/value pairs in order to be able to compose several map/reduce instances
- Keys and values themselves could be complex objects (including tuples).
- Map and Reduce functions are written by programmer
- Number of map tasks and reduce tasks given at start of program
- The rest done automatically (at least conceptually)

Example: Word Count

- Given: Document Set DS(K, documenttext)
- Output: For each word *w* occurring at least in one document of DS: indicate the number of occurrences of *w* in DS

Word Count


userdefined
map()
userdefined
reduce()
hi
hello
hello
hey
hi
Map
<hello, 1>
<hey, 1>
<hi, 1>
<hello, 1>
<hi, 1>
…
<hey, [1, 1]>
<hello, [1,1,1]>
Reduce
<hey, 2>
<hello, 3>
hey
hello
hi
hi
Map
<hi, 1>
<hi, 1>
<hello, 1>
<hey, 1>
…
<hi, [1, 1,1,1]>
Reduce
<hi, 4>

Map Step

- Input Parameters from User
  - Number m of map tasks
  - Number r of reduce tasks
  - Data set = document set DS
- Map function written by User

  WordCountMap:

  For each input key/value pair (dkey, dtext)

  For each word w of dtext

  Output key-value pair (w, 1)

- System splits input set into m partitions
- System creates m map tasks, gives each one partition
- Each map task executes map function on its partition
- Map step only completes once all map tasks are done

Shuffle and Reduce Steps

- System sorts map outputs by key and transforms all key/value pairs $(k, v_1), (k, v_2), \ldots (k, v_n)$ with same key *k* to one key/value-list pair $(k, (v_1, v_2, \ldots v_n))$
  - For Word count: all ('hello', 1), ('hello', 1), ('hello', 1) … are transformed into one ('hello', (1,1,1,….))
- System partitions output by key into *r* partitions
- System creates *r* reduce tasks and assigns each one partition
- Each reduce task executes user written reduce function
- Example WordCountReduce:

  For each input key/value-list pair (k, (v_1, v_2, … v_n))

  Output (k, sum(v_1, v_2, … v_n))

Phase Details

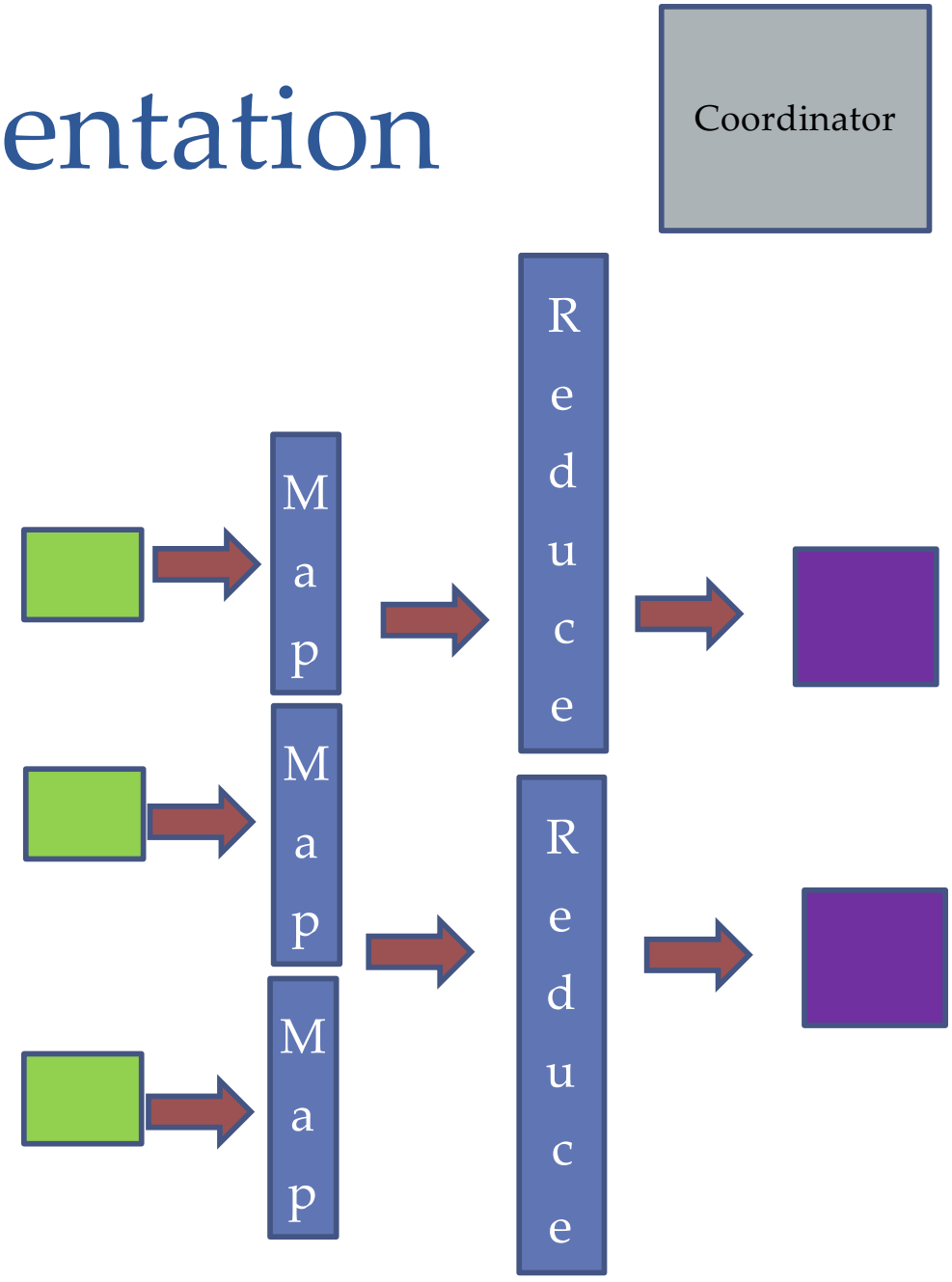
- Split into partitions
- At map tasks
  - Record reader
  - Map function
  - Possibly combine (explain later)
  - Write to local file
- Group and shuffle
  - Group keys and aggregate value-lists
  - Copy from map location to reduce location
- At reduce tasks
  - Reduce function and write to file system

Optional: Combine

- Possible if reduce function commutative and associative
- Execute reduce function at each mapper on partial result of mapper
- Reduces data to be transferred by shuffle
- Example word count:
  - At each mapper: count the occurrences of each word for all documents read by this mapper
  - For each mapper, there is then only one key-value pair for each word and the value is the number of occurrence
  - <hello, 1>, <hello, 1> → <hello, 2>

Implementation

- There is one coordinator node controlling execution
- Assigns tasks to map and reduce nodes at appropriate times
- Detects failures and performs **partial** restart



Selection with Map/reduce

- Assume R(A, B, C, D) relation (no duplicates)
- Selection with condition c on R
  - Map:
    - for each tuple t – (a,(b,c,d)) of R for which condition c holds, output (a,(b,c,d))
  - Reduce:
    - identity, that is, output (a,(b,c,d))

SELECT * FROM Users WHERE experience = 10

key

value

	uid	uname	experience	age
C_1	123	(Dora	2	13)
	132	(Bug	10	60)
C_2	267	(Sakura	10	15)
	111	(Cyphon	8	35)

Map

132	(Bug	10	60)
267	(Sakura	10	15)

Reduce

132	(Bug	10	60)
267	(Sakura	10	15)

Join with Map/reduce

- Natural Join R(A,B,C) with Q(C,D,E)
  - Map:
    - For each tuple (a,(b,c)) of R, output (c, (R, (a,b)))
    - For each tuple (c,(d,e)) of Q, output (c, (Q, (d,e)))
  - Group and shuffle will aggregate all key/value pairs with same c-value
  - Reduce

    For each tuple (c, value-list)

    (e.g., value-list = (R, (a1,b1)), (R, (a2,b2)),…(Q,(d1,e1)))

    Rt = Qt = empty;

    for each v=(rel,tuple) in value-list

    if v.rel = R: insert tuple into Rt else insert tuple into Qt

    for v1 in Rt, for v2 in Qt, output(c,(v1,v2))

    Basically produces all combinations (c, (ai,bi,dj,ej))

```
SELECT *
FROM Users  u, GroupMembers g
WHERE u.uid = g.uid
```

Users

uid	uname	experience	age
123	(Dora	2	13)
132	(Bug	10	60)
267	(Sakura	10	15)
111	(Cyphon	8	35)

GroupMembers

uid	gid	stars
(123	G1)	2
(132	G1)	5
(132	G2)	3
(132	G3)	1
(123	G2)	4
(111	G4)	2

Map

123	(U	(Dora	2	13))
132	(U	(Bug	10	60))
267	(U	(Sakura	10	15)
111	(U	(Cyphon	8	35)

123	(GM	(G1	2))
132	(GM	(G1	5))
132	(GM	(G2	3))
132	(GM	(G3	1))
123	(GM	(G2	4))
111	(GM	(G4	2))

```
SELECT *
FROM Users  u, GroupMembers g
WHERE u.uid = g.uid
```

123	(U	(Dora	2	13))
132	(U	(Bug	10	60))
267	(U	(Sakura	10	15)
111	(U	(Cyphon	8	35)

123	(GM	(G1	2))
132	(GM	(G1	5))
132	(GM	(G2	3))
132	(GM	(G3	1))
123	(GM	(G2	4))
111	(GM	(G4	2))

Shuffle

123	((U	(Dora	2	13))	(GM	(G1	2))	(GM	(G2	4)))	
132	((U	(Bug	10	60))	(GM	(G1	5))	(GM	(G2	3))	(GM	(G3	1)))
267	((U	(Sakura	10	15))							
111	((U	(Cyphon	8	35)	(GM	(G4	2)))				

Shuffle

123	((U	(Dora	2	13))	(GM	(G1	2))	(GM	(G2	4)))	
132	((U	(Bug	10	60))	(GM	(G1	5))	(GM	(G2	3))	(GM	(G3	1)))
267	((U	(Sakura	10	15))							
111	((U	(Cyphon	8	35)	(GM	(G4	2)))				

Reduce

123	(Dora	2	13	G1	2)
123	(Dora	2	13	G2	4)
132	(Bug	10	60	G1	5)
132	(Bug	10	60	G2	3)
132	(Bug	10	60	G3	1)
111	(Cyphon	8	35	G4	2)

U_T

Dora	2	13

GM_T

G1	2
G2	4

U_T

Bug	10	60

GM_T

G1	5
G2	3
G3	1

Discussion

- What to do when more complex query
  - Concatenate several map-reduce
- Generally:
  - Map-reduce paradigm not the best programming paradigm for many applications
- Advanced frameworks
  - Spark
    - Offers map function, offers reduce function
    - Offers specialized distributed SQL functions (filter, projection…) → see begin of lecture
    - Offers more efficient fault-tolerance
    - More flexible
  - ML frameworks